

Tiny Robotic Inference: Low-Latency LLM Planning for Robot Actions

Optimized Qwen2.5-3B JSON plan inference from 8.60s to 2.55s per request with vLLM prefix caching, mixed precision

Michael Mihaley · mjm2442

GitHub: <https://github.com/michael-mih/tiny-inference>
Tracking: Weights & Biases project [tiny-inference-latency](#)

Motivation & Problem

Low-latency LLM planning for robotics.

Why this matters

- Robot tasks need responsive language-to-action planning.
- Multi-second LLM decode delays slow demos and operator interaction.
- Naive HF Transformers fp32 generation took 8.60s per warmed request.

Problem statement

Optimize Qwen2.5-3B-Instruct JSON plan inference for batch-1 robot prompts on a Tesla T4.

Goal: Target $\geq 2\times$ lower latency at same JSON schema validity; achieved 3.38x vs fp32 baseline.

Technical Approach — Model & Data

Batch-1 inference workload; no model training.

Model

Qwen2.5-3B-Instruct, decoder-only; HF Transformers baseline and vLLM optimized path.

Dataset

Prompt/schema workload:
natural-language pick/place tasks ->
JSON plan with PICK, MOVE_TO, PLACE;
146 input tokens in final benchmark.

Hardware

1x NVIDIA Tesla T4 16GB, Debian 11,
PyTorch 2.6.0+cu124, CUDA 12.4,
Transformers, vLLM.

Technical Approach — Optimizations Applied

Optimizations tested and kept when they improved latency.

1. Mixed Precision

Reduced precision lowered memory bandwidth and model footprint; best Hugging Face path used bf16/fp16.

2. vLLM + Prefix Caching

Optimized serving/decode path and reused the static prompt prefix across repeated requests.

3. Prompt / Generation Trimming

Shorter schema prompt, no repair pass by default, and JSON-aware stopping reduced token work.

Profiling Methodology

Steady-state inference only: model loaded, prompt prepared, warmups complete.

Tools

- Custom benchmark.py with CUDA synchronize around generate()
- vLLM engine logs and scheduler configuration
- nvidia-smi / NVML telemetry where available
- Weights & Biases for run-level tracking

Metrics captured

- Average, p50, p95 latency; output tokens/sec
- Average GPU utilization
- Peak GPU memory
- Setup latency vs warmed generation latency
- JSON schema validity and ROS plan expansion

Results — Headline Numbers

Measured final run: fp32 Transformers baseline vs optimized vLLM prefix caching.

3.38x

faster inference

8.60s -> 2.55s avg latency

1.49x

faster than HF mixed precision

3.80s -> 2.55s avg latency

+1.6%

peak GPU memory tradeoff

11.97GB -> 12.16GB

10 timed runs, 3 warmups, batch=1 on Tesla T4; optimized uses vLLM fp16 prefix caching.

Results — Before vs. After

Measured rows from W&B latency_reports table. <https://wandb.ai/mjm2442-columbia-university/tiny-inference-latency/runs/6enskkzu?nw=nwusermjm2442>

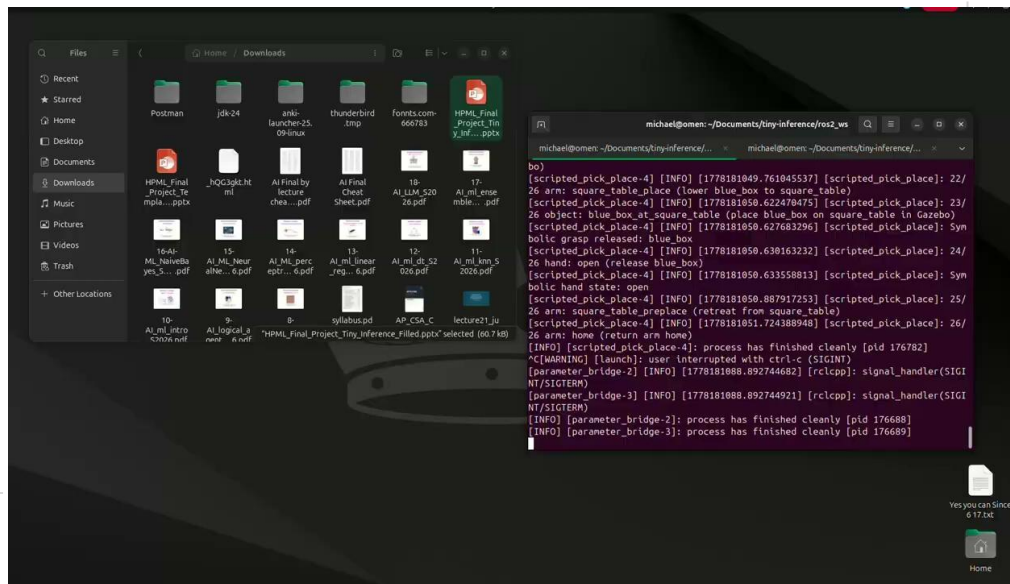
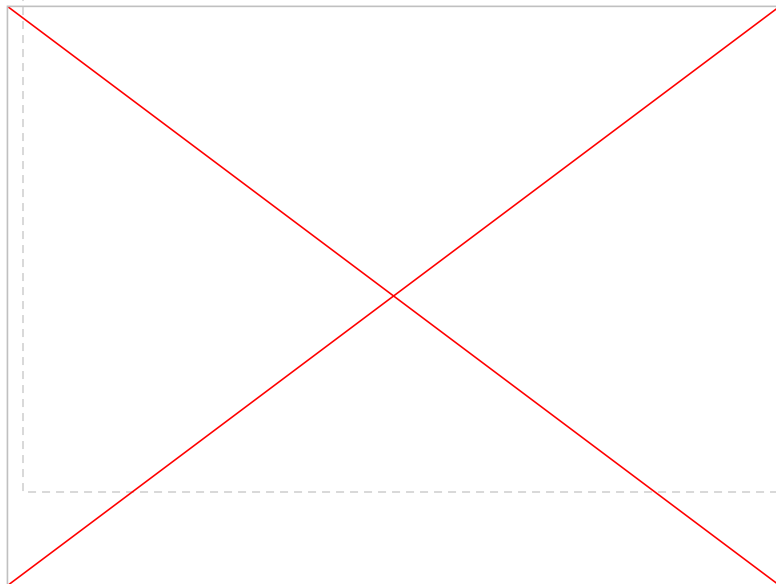
Takeaway

- fp32 Transformers: 8601 ms; HF mixed precision: 3805 ms; optimized vLLM prefix caching: 2547 ms.
- Largest final gain came from vLLM + prefix caching after mixed precision plateaued.
- Tradeoff: vLLM setup was ~34.9s and peak VRAM increased slightly; persistent server amortizes this.

Demo / Visualization

Demo path: optimized prompt server feeds the ROS/Gazebo pick-place node.

Instruction -> JSON plan -> ROS plan_schema expansion -> pick/place in Gazebo.



Lessons Learned

What worked, what failed, and what changed after profiling.

What worked

- vLLM prefix caching was the best latency improvement for this batch-1 serving setup.
- Persistent prompt server matters: setup cost is high but amortized over requests.
- Measuring output tokens exposed hidden latency from over-large generation caps.

What didn't work

- `torch.compile` was essentially flat for this short autoregressive workload.
- `bitsandbytes` quantization regressed latency
- vLLM defaults assumed high concurrency; T4 needed `fp16` and `max_num_seqs=1`.

Next Steps

Concrete follow-ups to improve latency and robustness.

- Benchmark Qwen2.5-1.5B/0.5B or distill the planner to a smaller model.
- Add constrained JSON / grammar decoding in vLLM to guarantee validity and stop earlier.
- Evaluate on a larger set of robot instructions and measure real ROS end-to-end latency.

Thank you

Questions?

GitHub: <https://github.com/michael-mih/tiny-inference>

Tracking: Weights & Biases project `tiny-inference-latency`